

Reviewer Pack — Minimal Rank of Quantum Tensor Network and Resolution Proof ...

Entry 555989a0185b · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-07 09:30:43 UTC

Minimal Rank of Quantum Tensor Network and Resolution Proof Width

Entry ID: 555989a0185b **Verdict:** INCONCLUSIVE **Recorded:** 2026-06-07 09:30:43 UTC

1. Conjecture

Field A (mathematical branch): Quantum Information Theory (Quantum Tensor Networks) **Field B** (complexity object): Boolean Satisfiability (Resolution Proof Complexity)

Statement:

The minimal rank of the quantum tensor network associated with a given CNF φ is linearly correlated with its resolution proof width $w(\varphi)$, such that $R_{\min}(\varphi) = \Theta(w(\varphi))$ where $R_{\min}(\varphi)$ denotes the minimum rank of the quantum tensor network.

Rationale (proposer's reasoning):

Quantum tensor networks provide a representation for quantum states, and their ranks can encode information about the complexity of certain computations. This conjecture suggests a potential connection between the structure of quantum states and the complexity of resolving boolean satisfiability problems.

Taxonomy category: Quantum_Complexity (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 5806c71da5716e9e

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

The conjecture is supported if the correlation coefficient between the minimal rank of the quantum tensor network ($R_{\min}(\varphi)$) and the resolution proof width ($w(\varphi)$) for all test cases is ≥ 0.8 , and the mean absolute difference between $R_{\min}(\varphi)$ and $w(\varphi)$ over all seeds is ≤ 3 .

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	UNCERTAIN	1.00	HITS	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=124, elapsed=240.3s

5.1 Generated Python source

```
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_cnf(n, m):
        cnf = []
        for _ in range(m):
            clause = [random.randint(1, n), -random.randint(1, n)]
            cnf.append(clause)
        return cnf

    def construct_qtn(cnf, n):
        qtn = [[0] * (n + 1) for _ in range(n + 1)]
        for clause in cnf:
            var1, var2 = abs(clause[0]), abs(clause[1])
            if clause[0] > 0 and clause[1] > 0:
                qtn[var1][var2] += 1
                qtn[var2][var1] += 1
            elif clause[0] < 0 and clause[1] < 0:
                qtn[-var1][-var2] += 1
                qtn[-var2][-var1] += 1
        return qtn

    def matrix_rank(matrix):
        m, n = len(matrix), len(matrix[0])
        rank = min(m, n)
        for i in range(rank):
            if matrix[i][i] == 0:
                found = False
                for j in range(i + 1, m):
                    if matrix[j][i] != 0:
                        matrix[i], matrix[j] = matrix[j], matrix[i]
                        found = True
                        break
            if not found:
```

```

        rank -= 1
        continue
    for j in range(n):
        if j != i and matrix[i][j] != 0:
            factor = -matrix[j][i] / matrix[i][i]
            for k in range(i, n):
                matrix[j][k] += factor * matrix[i][k]
    return rank

def resolution_width(cnf):
    clauses = set(tuple(sorted(clause)) for clause in cnf)
    width = 0
    while len(clauses) > 1:
        new_clauses = set()
        for clause1, clause2 in itertools.combinations(clauses, 2):
            if not (set(clause1) & set(clause2)):
                new_clause = tuple(sorted(set(clause1) | set(clause2)))
                if len(new_clause) > width:
                    width = len(new_clause)
                new_clauses.add(new_clause)
        clauses = new_clauses
    return width

n = random.randint(5, 30)
m = random.randint(2 * n, 4 * n)
cnf = generate_cnf(n, m)

qtn = construct_qtn(cnf, n)
rank = matrix_rank(qtn)
width = resolution_width(cnf)

return {
    "metric_name": "Rank vs Width",
    "metric_value": abs(rank - width),
    "instances_tested": 1,
    "n_max": n,
    "conjecture_holds": abs(rank - width) <= 3,
    "counterexample": ""
}

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] if sys.argv[1:] else [
        2, 3, 5, 7, 11, 13, 17, 19, 23, 29,
        31, 37, 41, 43, 47, 53, 59, 61, 67, 71,
        73, 79, 83, 89, 97, 101, 103, 107, 109, 113
    ]

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {{\"seed\": {seed}, \"metric_name\": \"{result['metric_name']}\", \"metric_
        results.append(result)

    mean_value = sum(r["metric_value"] for r in results) / len(results)
    std_value = math.sqrt(sum((r["metric_value"] - mean_value) ** 2 for r in results) / len(results))

```

```

support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

if all(r["conjecture_holds"] for r in results):
    print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_fraction}")
elif support_fraction >= 0.8:
    print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_fraction}")
else:
    for r in results:
        if not r["conjecture_holds"]:
            counterexample = f"n={r['n_max']}, rank={matrix_rank(construct_qtn(generate_cnf(r)))}"
            print(f"RESULT: FALSIFIED counterexample=\"{counterexample}\" first_failing_seed={r['seed']}")
            break

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

TIMEOUT after 240s

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test timed out before producing data, which means that the correlation coefficient and mean absolute difference could not be calculated. | next: NONE

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	14017
2	preregistration	ollama_remote	glm4:latest	0	0	9411
3	novelty	ollama_remote	glm4:latest	0	0	8423
4	novelty	ollama_remote	glm4:latest	0	0	8473
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	44118
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11137
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12794
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	14206
9	judge	ollama_remote	glm4:latest	0	0	23216

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 145795 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in `research/audit/555989a0185b.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/555989a0185b.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/555989a0185b.tar.gz` (if generated)